

Creating elements from an array in JavaScript

Let's have some array:

```
1 | let arr = [1, 2, 3, 4, 5];
```

Let also we have some parent element:

```
1 | <div id="parent"></div>
```

Let's add new paragraphs to our parent, the text of which will be the elements of our array.

Implementing the described:

```
1 | let parent = document.querySelector('#parent');
2 |
3 | let arr = [1, 2, 3, 4, 5];
4 |
5 | for (let elem of arr) {
6 |   let p = document.createElement('p');
7 |   p.textContent = elem;
8 |
9 |   parent.appendChild(p);
10 | }
```

№ 1

⊗ jsPmPrECVA

Modify my code so that when you click on a paragraph, one is added to its content.

Practice on creating ul lists in JavaScript

№ 1

⊗ jsPmPrLC

Given an `ul`:

```
1 | <ul id="elem"></ul>
```

Given an array. Insert the elements of this array at the end of `ul` so that each element is in its own `li`.

№ 2

⊗ jsPmPrLC

Modify the previous task so that when you click on any of the inserted elements, the text of this element is displayed on the screen.

№ 3

⊗ jsPmPrLC

Modify the previous task so that clicking on a `li` adds '!' to its end.

№ 4

⊗ jsPmPrLC

Modify the previous task so that pressing the `li` again doesn't add a second '!'.

Creating HTML tables in JavaScript

Suppose we have such an empty HTML table:

```
1 <table id="table"></table>
```

Let's fill this table with rows and columns. Here is an example of what we should obtain:

```
1 <table id="table">
2   <tr>
3     <td></td>
4     <td></td>
5     <td></td>
6   </tr>
7   <tr>
8     <td></td>
9     <td></td>
10    <td></td>
11  </tr>
12  <tr>
13    <td></td>
14    <td></td>
15    <td></td>
16  </tr>
17 </table>
```

To solve the problem, we need two nested loops. The first loop will create the rows of the table and the second will create the cells in each row:

```
1 let table = document.querySelector('#table');
2
3 for (let i = 0; i < 3; i++) {
4   let tr = document.createElement('tr');
5
6   for (let i = 0; i < 3; i++) {
7     let td = document.createElement('td');
8     tr.appendChild(td);
9   }
10
11   table.appendChild(tr);
12 }
```

№ 1

⊗jsPmPrTC

Given an empty HTML table. Fill this table with 5 rows with 5 columns using two nested `for` loops.

№ 2

⊗jsPmPrTC

Modify the previous task so that the table is 10 rows by 5 columns.

№ 3

⊗jsPmPrTC

Modify the previous task so that the text 'x' is added to each `td`.

Sequential populating an HTML table

Suppose we have such an empty HTML table:

```
1 <table id="table"></table>
```

Let's populate this table with cells and make these cells contain numbers from 1 to 9. Here is an example of what we should get:

```
1 <table id="table">
2   <tr>
3     <td>1</td>
4     <td>2</td>
5     <td>3</td>
6   </tr>
7   <tr>
8     <td>4</td>
9     <td>5</td>
10    <td>6</td>
11  </tr>
12  <tr>
13    <td>7</td>
14    <td>8</td>
15    <td>9</td>
16  </tr>
17 </table>
```

Let's now make it so that instead of the letters 'x', numbers are written into the cells in ascending order.

To do this, we need to introduce another counter, which will sequentially increase its values in each iteration of the inner loop, like this:

```
1 let table = document.querySelector('#table');
2
3 let k = 1; // a counter initial value
4 for (let i = 0; i < 3; i++) {
5   let tr = document.createElement('tr');
6
7   for (let i = 0; i < 3; i++) {
8     let td = document.createElement('td');
9
10    td.textContent = k; // writes the counter to the cell
11    k++; // increments the counter
12
13    tr.appendChild(td);
14  }
15
16  table.appendChild(tr);
17 }
```

№1

©jsPmPrTCF

Display an HTML table of 5 rows by 5 columns so that its cells contain consecutive numbers from 1 to 25.

№2

©jsPmPrTCF

Modify the previous task so that even numbers in the interval from 2 to 50 are written in the table cells.

Let's get started with the implementation.

First, let's just make a table of 3 by 3 size, filled with letters 'x':

```
1 let table = document.querySelector('#table');
2
3 for (let i = 0; i < 3; i++) {
4   let tr = document.createElement('tr');
5
6   for (let i = 0; i < 3; i++) {
7     let td = document.createElement('td');
8     td.textContent = 'x';
9     tr.appendChild(td);
10  }
11
12  table.appendChild(tr);
13 }
```

Creating an HTML table from an array in JavaScript

Suppose we have some empty table:

```
1 <table id="table"></table>
```

Suppose we also have some two-dimensional array, for example, this:

```
1 let arr = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]];
```

Let's make an HTML table based on this array, filled with the elements of this array. That is, we will get a table with three rows, each row will have 4 cells.

Solve the problem with two nested `for-of` loops:

```
1 let arr = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]];
2 let table = document.querySelector('#table');
3
4 for (let subArr of arr) {
5   let tr = document.createElement('tr');
6
7   for (let elem of subArr) {
8     let td = document.createElement('td');
9     td.textContent = elem;
10    tr.appendChild(td);
11  }
12
13  table.appendChild(tr);
14 }
```

As you can see, our solution turned out to be universal and does not depend on the number of subarrays and the number of elements in each subarray. The only condition is that the subarrays have the same number of elements.

№ 1

©jsPmPrTCVA

Given an array:

```
1 let arr = [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]];
```

Without looking into the theoretical part of the lesson, output the elements of the given array as an HTML table `table`.

№ 2

©jsPmPrTCVA

Modify the previous task so that the table doesn't contain elements, but the squares of these elements.

Creating an HTML table from an array of objects

Let's say we have some array of objects, for example, here's an array of users:

```
1  let users = [  
2    {  
3      name: 'name1',  
4      surname: 'surname1',  
5      patronymic: 'patronymic1'  
6    },  
7    {  
8      name: 'name2',  
9      surname: 'surname2',  
10     patronymic: 'patronymic2'  
11   },  
12   {  
13     name: 'name3',  
14     surname: 'surname3',  
15     patronymic: 'patronymic3'  
16   },  
17 ];
```

Let's make a HTML table from this array, in each row we will write the data of an individual user.

With this data storage structure, they usually run one loop through the array and create each cell of the table manually, like this:

```
1 <table id="table"></table>

1 let table = document.getElementById('table');
2
3 for (let user of users) {
4   let tr = document.createElement('tr');
5
6   let td1 = document.createElement('td');
7   td1.textContent = user.name;
8   tr.appendChild(td1);
9
10  let td2 = document.createElement('td');
11  td2.textContent = user.surname;
12  tr.appendChild(td2);
13
14  let td3 = document.createElement('td');
15  td3.textContent = user.patronymic;
16  tr.appendChild(td3);
17
18  table.appendChild(tr);
19 }
```

This approach gives great flexibility - we ourselves can regulate the order of the data in the table cells (we can, for example, swap the first and last names).

In addition, if desired, we can hang events on certain cells. For example, you can attach some kind of action on click to the cell with the last name, and so on.

№ 1

©jsPmPrTCVOA

Given the following array with employees:

```
1 let employees = [
2   {name: 'employee1', age: 30, salary: 400},
3   {name: 'employee2', age: 31, salary: 500},
4   {name: 'employee3', age: 32, salary: 600},
5 ];
```

Output this array elements as an HTML table.

№ 2

©jsPmPrTCVOA

Modify the previous task so that when you click on any cell with an age, its content increases by 1.

№ 3

©jsPmPrTCVOA

Modify the previous task so that when you click on any cell with a salary, its content increases by 10%.

Adding rows and columns to an HTML table

Let's say we have some HTML table - `#table`. Let's learn how to add new rows and columns to it.

Adding rows

Adding rows is not difficult: you need to create `tr`, and then run a loop that will add the desired number of cells to this row (let's say 3):

```
1 let table = document.querySelector('#table');
2
3 let tr = document.createElement('tr');
4
5 for (let i = 1; i <= 3; i++) {
6   let td = document.createElement('td');
7   tr.appendChild(td);
8 }
9
10 table.appendChild(tr);
```

№ 1

⊗jsPmPrTRCA

Make a button, by clicking on which a new row will be added to the table.

Adding columns

But with adding columns, it's a little more complicated: you need to run a loop that will go through all the table rows and add a new cell to each row:

```
1 let trs = document.querySelectorAll('#table tr');
2
3 for (let tr of trs) {
4   let td = document.createElement('td');
5   tr.appendChild(td);
6 }
```

№ 2

⊗jsPmPrTRCA

Given a 2 by 2 table:

```
1 <table id="table">
2   <tr>
3     <td></td>
4     <td></td>
5   </tr>
6   <tr>
7     <td></td>
8     <td></td>
9   </tr>
10 </table>
```

```
1 td {
2   width: 50px;
3   height: 50px;
4   border: 1px solid black;
5 }
```

There is also a button. Make it so that when you click on the button, the table increases by one row and one column.

Modifying HTML table cells in JavaScript

Let some HTML table `#table` be given. Suppose we are faced with the task of doing something with each cell of the table, for example, writing the text `!` into each of them.

Let's discuss the nuances of solving such a problem.

As you know, an HTML table has a two-dimensional structure: there are rows, and there are cells in them. We can solve our problem as follows: loop through the rows and loop through the cells in each row and do the action we need with them. That is, the above solution scheme is similar to how we would create such a table, filling it with rows and cells.

However, in this case, two nested loops will be superfluous: you can simply get all `td` and loop through them, performing the desired operation. Let's do it:

```
1 let tds = document.querySelectorAll('#table td');
2
3 for (let td of tds) {
4   td.textContent = '!';
5 }
```

№ 1

⊗jsPmPrTCM

Let some HTML table with numbers and a button be given. By clicking on the button, double the number in each cell of the table.

Self-removal of new elements in JavaScript

In the previous lesson, we learned how to make elements remove themselves on click.

Let now there are no elements in the parent initially:

```
1 <div id="parent"></div>
```

Let's create 9 new paragraphs in a loop, making any paragraph removed by clicking on it:

```
1 let parent = document.querySelector('#parent');
2
3 for (let i = 1; i <= 9; i++) {
4   let p = document.createElement('p');
5   p.textContent = i;
6
7   p.addEventListener('click', function() {
8     this.remove();
9   });
10
11   parent.appendChild(p);
12 }
```

№ 1

⊗jsPmPrNER

Given the following code:

```
1 <ul id="parent">
2   <li>1</li>
3   <li>2</li>
4   <li>3</li>
5 </ul>
6
7 <input type="submit" id="button">
```

Make it so that when you click on the button, a new element is added to the list. Make sure that any `li` is removed by clicking on it. We are talking about both those `li` that are already in the list, as well as new ones created after pressing the button.

Link to remove an element in JavaScript

Let's have a paragraph. Let's make a link next to it, with the help of which this paragraph can be deleted.

Let's implement:

```
1 <div id="parent">
2   <p id="elem">text</p>
3   <a href="#" id="remove">remove</a>
4 </div>

1 let elem = document.querySelector('#elem');
2 let remove = document.querySelector('#remove');
3
4 remove.addEventListener('click', function() {
5   elem.remove();
6 });
```

Note that the link has a # in the href attribute. If this hash is removed, we will get a link transition and, as a result, a page refresh.

In fact, the deletion of the paragraph will also occur, but we will not notice this, since the page will be refreshed and everything will return to its original state.

To solve the problem, you need to prevent following the link with `preventDefault`:

```
1 <div id="parent">
2   <p id="elem">text</p>
3   <a href="#" id="remove">remove</a>
4 </div>

1 let elem = document.querySelector('#elem');
2 let remove = document.querySelector('#remove');
3
4 remove.addEventListener('click', function(event) {
5   elem.remove();
6   event.preventDefault(); // cancels the link transition
7 });
```

№ 1

⊗jsPmPrRL

On your own, without looking into my code, solve the described problem.

Creating links to remove elements in JavaScript

Let now we have many paragraphs:

```
1 <div id="parent">
2   <p>text1</p>
3   <p>text2</p>
4   <p>text3</p>
5 </div>
```

Let's make sure that each paragraph has a link at the its end to remove it.

To get started, let's just implement adding links:

```
1 let elems = document.querySelectorAll('#parent p');
2
3 for (let elem of elems) {
4   let remove = document.createElement('a');
5   remove.href = '';
6   remove.textContent = 'remove';
7
8   elem.appendChild(remove);
9 }
```

Let's now make it so that when you click on the link, the corresponding paragraph is removed:

```
1 let elems = document.querySelectorAll('#parent p');
2
3 for (let elem of elems) {
4   let remove = document.createElement('a');
5   remove.href = '';
6   remove.textContent = 'remove';
7   elem.appendChild(remove);
8
9   remove.addEventListener('click', function(event) {
10     elem.remove();
11     event.preventDefault();
12   });
13 }
```

№ 1

⊗jsPmPrCRL

The `ul` tag is given. Add a link to the end of each `li` tag to remove that `li` from the list.

№ 2

⊗jsPmPrCRL

Given an HTML table. Add another column to it, in which for each row of the table there will be a link to remove this row.

Editing a single element in JavaScript

Now we will learn how to edit element texts using input fields. Let's start with simple things and gradually get more complex.

So, let's say we have a paragraph and an input located in the same parent:

```
1 <div id="parent">
2   <p id="elem">text</p>
3   <input id="input">
4 </div>
```

Let's make it so that when the focus is lost in the input, its text appears in the paragraph:

```
1 let elem = document.querySelector('#elem');
2 let input = document.querySelector('#input');
3
4 input.addEventListener('blur', function() {
5   elem.textContent = this.value;
6 });
```

Let's now make it so that when you enter the page, the input already contains the text of the paragraph. Thus, using the input, we will be able to edit the text located in the paragraph.

Let's implement:

```
1 let elem = document.querySelector('#elem');
2 let input = document.querySelector('#input');
3
4 input.value = elem.textContent; // writes the paragraph text to the input
5
6 input.addEventListener('blur', function() {
7   elem.textContent = this.value;
8 });
```

Modify the code above so that the paragraph text doesn't change when focus is lost, but as text is entered into the input.

Appearance of input

Let's now make it so that the input was not initially on the page, but it appeared when clicking on the paragraph. So our initial HTML will look like this:

```
1 <div id="parent">
2   <p id="elem">text</p>
3 </div>
```

To begin with, we simply implement the appearance of the input, without editing:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function() {
4   let input = document.createElement('input');
5   input.value = elem.textContent;
6
7   elem.parentElement.appendChild(input);
8 });
```

And now let's make it so that when the focus is lost in the input, the text of the paragraph changes:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function() {
4   let input = document.createElement('input');
5   input.value = elem.textContent;
6
7   input.addEventListener('blur', function() {
8     elem.textContent = this.value;
9   });
10
11   elem.parentElement.appendChild(input);
12 });
```

Our code, however, is not perfect, since each click on a paragraph will result in a new input.

To solve the problem, we will simply remove the current input by losing focus:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function() {
4   let input = document.createElement('input');
5   input.value = elem.textContent;
6
7   input.addEventListener('blur', function() {
8     elem.textContent = this.value;
9     this.remove(); // removes the input
10  });
11
12   elem.parentElement.appendChild(input);
13 });
```

On your own, without looking into my code, solve the described problem.

Hiding text while editing an element in JavaScript

Let's now make it so that when editing, the input appears in the paragraph itself - instead of the text of this paragraph. Let then, at the end of editing, the input will be removed, and instead of it, the paragraph text will appear again.

Let's start implementation.

First, let's make it so that when you click on the paragraph, an input with the text of this paragraph is added to the end of it:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function() {
4   let input = document.createElement('input');
5   input.value = elem.textContent;
6
7   elem.appendChild(input);
8 });
```

Our code, however, is very imperfect - every time you click on a paragraph, a new input will be added to it.

In this case, if the first input contains the text of the paragraph, then the second input will already contain the text of the paragraph along with the first input, and the third input will already contain the text of the paragraph along with two inputs, and so on.

Also note that clicking on the added input will be treated as a click on the paragraph: the fact is that the input is in the paragraph, and clicking on the input will simply float above this paragraph.

This will lead to the fact that after the appearance of the first input when trying to click on it in order to start editing, we will automatically click on the paragraph with all the ensuing consequences.

So, the problem is described. Let's fix it now.

To do this, simply unbind the click handler from the paragraph when an input appears. In this case, only the first click on the paragraph will result in an input, and other clicks after the input appearance will be ignored.

Let's implement:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function func() {
4   let input = document.createElement('input');
5   input.value = elem.textContent;
6
7   elem.appendChild(input);
8
9   elem.removeEventListener('click', func); // unbinds the event
10 });
```

Let's now make it so that when an input appears, the text of the paragraph itself disappears. To do this, before inserting the input of the paragraph, assign an empty string to `textContent`:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function func() {
4   let input = document.createElement('input');
5
6   input.value = elem.textContent; // first writes the text of the paragraph to the input
7   elem.textContent = ''; // then removes the paragraph text
8   elem.appendChild(input); // then inserts the input
9
10  elem.removeEventListener('click', func);
11 });
```

Let's now make it so that when focus is lost in the input, its text is written in a paragraph:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function func() {
4   let input = document.createElement('input');
5
6   input.value = elem.textContent;
7   elem.textContent = '';
8   elem.appendChild(input);
9
10  input.addEventListener('blur', function() {
11    elem.textContent = this.value;
12  });
13
14  elem.removeEventListener('click', func);
15 });
```

Note that we don't need to remove the input - it removes itself when it writes its text to the paragraph: since the input is part of the text of the paragraph, writing some text to this paragraph simply removes our input, and that's it.

However, we have another problem: the text of the paragraph will only be edited the first time. After the first edit, clicking the paragraph text again will do nothing.

The fact is that at the moment the input appeared, we unbound the event from the paragraph for the reasons described above. Now we need to bind the event back at the end of editing.

Let's do it:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function func() {
4   let input = document.createElement('input');
5   input.value = elem.textContent;
6
7   elem.textContent = '';
8   elem.appendChild(input);
9
10  input.addEventListener('blur', function() {
11    elem.textContent = this.value;
12    elem.addEventListener('click', func); // hangs the event back
13  });
14
15  elem.removeEventListener('click', func);
16 });
```

Editing in a group of elements in JavaScript

Let now we have not single paragraph, but many:

```
1 <p>text1</p>
2 <p>text2</p>
3 <p>text3</p>
```

Let's make it so that when you click on any paragraph, an input for editing appears in it.

In fact, such a task is not difficult for us, since almost all the code was obtained by us in the previous lesson.

To solve our problem, we simply run a loop through paragraphs and use the code from the previous lesson in the loop (this code doesn't even need to be changed):


```

1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', function func() {
5     let input = document.createElement('input');
6     input.value = elem.textContent;
7
8     elem.textContent = '';
9     elem.appendChild(input);
10
11    input.addEventListener('blur', function() {
12      elem.textContent = this.value;
13      elem.addEventListener('click', func);
14    });
15
16    elem.removeEventListener('click', func);
17  });
18 }

```

No 1

©jsPmPrEGE

The `ul` tag is given. Make it so that by clicking on any `li` an input appears in it, using it you can edit the text of this `li`.

Simultaneous editing and removal of elements

Suppose we have some set of paragraphs:

```

1 <div id="parent">
2   <p>text1</p>
3   <p>text2</p>
4   <p>text3</p>
5 </div>

```

Let's make these paragraphs so that you can edit the text of each paragraph and at the same time at the end of each paragraph there was a link to remove.

When implementing the described task, we are faced with a certain problem.

In order to understand the essence of the problem, let's look at the HTML code that will result when links to remove are added to the end of each paragraph:

```

1 <div id="parent">
2   <p>text1<a href="">remove</a></p>
3   <p>text2<a href="">remove</a></p>
4   <p>text3<a href="">remove</a></p>
5 </div>

```

Imagine now that by clicking on any paragraph, an input for editing the text will appear in it. In this case, the entire text of the paragraph will be included in the input - along with a link to remove!

This is of course not correct.

A better solution would be to wrap paragraph texts in `span` tags, like this:

```

1 <div id="parent">
2   <p><span>text1</span><a href="">remove</a></p>
3   <p><span>text2</span><a href="">remove</a></p>
4   <p><span>text3</span><a href="">remove</a></p>
5 </div>

```

For such code, you can simply hang an editing event not on the paragraph itself, but on `span` with text. In this case, the edit input will appear in the `span` tag, and our removal link will remain intact.

No 1

© jsPmPrRET

Given the following HTML code:

```
1 <div id="parent">
2   <p><span>text1</span></p>
3   <p><span>text2</span></p>
4   <p><span>text3</span></p>
5 </div>
```

Add a removal link at the end of each paragraph.

Make it so that by clicking on `span` an input for editing appears in it.

№2

©jsPmPrET

Now let there be no `span` tags initially:

```
1 <div id="parent">
2   <p>text1</p>
3   <p>text2</p>
4   <p>text3</p>
5 </div>
```

First wrap the text of the paragraph in `span` tags, add editability to these tags, and then add a removal link at the end of each paragraph.

Element styling in JavaScript

Let's have a few paragraphs:

```
1 <p>text1</p>
2 <p>text2</p>
3 <p>text3</p>
```

Let's go through these paragraphs in a loop and add a link to the end of each paragraph, by clicking on which some styling will be added to the text of the paragraph. For example, the text of a paragraph will become strikethrough (this is done by the CSS property `text-decoration`).

Let's discuss two aspects of this problem..

First, as already discussed in previous lessons, you should not change the styles of paragraphs directly through JavaScript - it will be much more convenient to hang some CSS classes.

Secondly, when implementing such a task, some surprise awaits you. To understand its essence, let's look at the HTML code of paragraphs after adding links:

```
1 <p>text1<a href="">link</a></p>
2 <p>text2<a href="">link</a></p>
3 <p>text3<a href="">link</a></p>
```

Imagine now that when you click on the link, the text of the paragraph is crossed out. However, the link in this case is also part of the paragraph and will also be crossed out! Most likely we do not need such an effect. We want the text to strike through, but the link doesn't.

To solve the problem, you just need to wrap the paragraph text in a `span` tag, like this:

```
1 <p><span>text1</span><a href="">link</a></p>
2 <p><span>text2</span><a href="">link</a></p>
3 <p><span>text3</span><a href="">link</a></p>
```


№ 1

© jsPmPrES

Given the following HTML code:

```
1 | <p>text1</p>
2 | <p>text2</p>
3 | <p>text3</p>
```

Add a link at the end of each paragraph, upon clicking on which the text of the paragraph will be crossed out (but the link isn't).

№ 2

© jsPmPrES

Modify the previous task so that after clicking on the link, this link is removed from the paragraph (and the text of the paragraph becomes strikethrough).

№ 3

© jsPmPrES

Given some HTML table. Add another column with a link to this table. By clicking on this link, the row with this link should turn green.

Buttons to hide and show an element in JavaScript

In this section, we will learn how to hide and show page elements. As usual, we will start with something simple, and we will gradually complicate.

Suppose we have a paragraph and two buttons to warm up:

```
1 | <p id="elem">text</p>
2 | <input type="submit" id="show" value="show">
3 | <input type="submit" id="hide" value="hide">
```

Let's get references to our elements into variables:

```
1 | let elem = document.querySelector('#elem');
2 | let show = document.querySelector('#show');
3 | let hide = document.querySelector('#hide');
```

Now let's make it so that by clicking on one button our paragraph is hidden, and on the other it is shown. To do this, we will add or remove the corresponding CSS class to the element:

```
1 | .hidden {
2 |   display: none;
3 | }
```

Let's solve our problem:

```
1 | hide.addEventListener('click', function() {  
2 |     elem.classList.add('hidden');  
3 | });  
4 |  
5 | show.addEventListener('click', function() {  
6 |     elem.classList.remove('hidden');  
7 | });
```

№ 1

©jsPmPrHShB

Modify my code so that there is only one button. Let the element be shown on the first click on this button, and hidden on the second.

Many elements with show buttons in JavaScript

Let now we have a lot of paragraphs and each has its own button to hide:

```
1 <p>1</p><button>toggle</button>
2 <p>2</p><button>toggle</button>
3 <p>3</p><button>toggle</button>
```

Let's make it so that clicking on the button hides or shows the paragraph corresponding to it.

To do this, we somehow need to associate the buttons with our paragraphs. There are several ways to do this.

First way

Let's link buttons and paragraphs as follows:

```
1 <p id="elem1">1</p><button data-elem="elem1">toggle</button>
2 <p id="elem2">2</p><button data-elem="elem2">toggle</button>
3 <p id="elem3">3</p><button data-elem="elem3">toggle</button>
```

Now, by clicking on any button, we will read the contents of its data-elem attribute and look for a paragraph with that id. We'll toggle it. Let's implement the described:

```
1 let buttons = document.querySelectorAll('button');
2
3 for (let button of buttons) {
4   button.addEventListener('click', function() {
5     let elem = document.querySelector('#' + this.dataset.elem);
6     elem.classList.toggle('hidden');
7   });
8 }
```

No 1

©jsPmPrMEWShB

Study my solution, and then independently, without looking at my code, solve this problem.

Second way

It is not very convenient to arrange id and data attributes. Let's make it so that the relationship is by ordinal number: let the first button hide the first paragraph, the second button - the second and so on.

Let's implement the described:

```
1 let buttons = document.querySelectorAll('button');
2 let elems = document.querySelectorAll('p');
3
4 for (let i = 0; i < buttons.length; i++) {
5   buttons[i].addEventListener('click', function() {
6     elems[i].classList.toggle('hidden');
7   });
8 }
```


№2

©jsPmPrMEWShB

Study my solution, and then independently, without looking at my code, solve this problem.

Third way

As you can see, the paragraph associated with a button is its left neighbor. This can be used as a link:

```
1 | let buttons = document.querySelectorAll('button');
2 |
3 | for (let button of buttons) {
4 |   button.addEventListener('click', function() {
5 |     this.previousElementSibling.classList.toggle('hidden');
6 |   });
7 | }
```

№3

©jsPmPrMEWShB

Study my solution, and then independently, without looking at my code, solve this problem.

Element activation in JavaScript

Suppose we have some HTML table #table with empty cells.

Let's make it so that when you click on any cell, it is somehow *activated*, for example, it gets a red background. To do this, we will add some CSS class to activated cells:

```
1 | .active {
2 |   background: red;
3 | }
```

Let's implement activation:

```
1 | let tds = document.querySelectorAll('#table td');
2 |
3 | for (let td of tds) {
4 |   td.addEventListener('click', function() {
5 |     this.classList.add('active');
6 |   });
7 | }
```

№1

©jsPmPrEA

Given an HTML list `ul`. Make it so that when you click on any item in the list, it is activated with a red background.

№2

©jsPmPrEA

Modify the previous task so that by clicking on the activated list item, activation is removed from it.

Alternation of activation styles in JavaScript

In the previous lesson, we learned how to activate cells by clicking on them. Let's now make the colors alternate: let the first cell be painted red, the second cell green, the third cell red again, and so on.

To solve this problem, we need two CSS classes already:

```
1 | .color1 {  
2 |   background: red;  
3 | }  
4 | .color2 {  
5 |   background: green;  
6 | }
```

Let's implement the described alternation:

```
1 | let tds = document.querySelectorAll('#table td');  
2 |  
3 | let color = 'color1';  
4 | for (let td of tds) {  
5 |   td.addEventListener('click', function() {  
6 |     if (color == 'color1') {  
7 |       color = 'color2';  
8 |     } else {  
9 |       color = 'color1';  
10 |    }  
11 |  
12 |    this.classList.add(color);  
13 |   });  
14 | }
```

№1

©jsPmPrASA

Take a look at my code and then try this problem yourself.

Several colors alternation from an array

Let now we want to alternate not two colors, but an arbitrary number. Let's store an array of CSS classes for this:

```
1 | let colors = ['color1', 'color2', 'color3'];
```

Let's write the colors for our classes:

```
1 | .color1 {  
2 |   background: red;  
3 | }  
4 | .color2 {  
5 |   background: green;  
6 | }  
7 | .color3 {  
8 |   background: yellow;  
9 | }
```

And now let's implement alternation:

```
1 let tds = document.querySelectorAll('#table td');
2
3 let i = 0;
4 let colors = ['color1', 'color2', 'color3'];
5
6 for (let td of tds) {
7   td.addEventListener('click', function() {
8     this.classList.add(colors[i]);
9
10    i++;
11    if (i == colors.length) {
12      i = 0;
13    }
14  });
15 }
```

No2

⊗ jsPmPrASA

Tell me how my code works.

Practice on changing elements in JavaScript

No1

⊗ jsPmPrEChT

Given an array. Output its elements as a `ul` list.

No2

⊗ jsPmPrEChT

Modify the previous task so that by clicking on any `li` an input appears in it, with which you can edit it.

No3

⊗ jsPmPrEChT

Modify the previous task so that there is an input below the list, with which you can add a new element to the end of the `ul` list. Make the new `li` also editable.

No4

⊗ jsPmPrEChT

Modify the previous task so that at the end of each `li` there is a link 'remove', which will remove this `li` from `ul`.

№5

⊗ jsPmPrEChT

Modify the previous task so that at the end of each `li` there is also a link 'cross out', which can be used to cross out the text of this `li` tag.

Array of objects and a table

№6

⊗ jsPmPrEChT

Given the following array with employees:

```
1 | let employees = [  
2 |   {name: 'employee1', age: 30, salary: 400},  
3 |   {name: 'employee2', age: 31, salary: 500},  
4 |   {name: 'employee3', age: 32, salary: 600},  
5 | ];
```

Display these employees in an HTML table.

№7

⊗ jsPmPrEChT

Add the ability to edit the cells of the created table.

№8

⊗ jsPmPrEChT

Add a new column to your table with a link to remove a row from the table.

№9

⊗ jsPmPrEChT

Make 3 inputs and a button for adding a new employee under the table. Let the name, age and salary be entered into the inputs, and by clicking on the button a new employee is added to the table. Implement cell editing for newly added employees.

Array of objects and a list

№ 10

©jsPmPrEChT

Given the following array with employees:

```
1 | let employees = [  
2 |   {name: 'employee1', age: 30, salary: 400},  
3 |   {name: 'employee2', age: 31, salary: 500},  
4 |   {name: 'employee3', age: 32, salary: 600},  
5 | ];
```

Display each employee in own `li` tag of the `ul` tag.

№ 11

©jsPmPrEChT

Make it so that when clicking on the name, age or salary of an employee, an input appears for editing this field.

№ 12

©jsPmPrEChT

Add a link to the end of each `li` tag to remove that `li` from the list.

№ 13

©jsPmPrEChT

Under the list, make a form for adding a new employee.